

***DSC 40B***

***Lecture 2 : Timing,  
Counting  
Operations, Nested  
Loop***

***Mic!***



# ***Announcements***

# Announcements

- Lab01 is posted on Gradescope.
  - Due Sunday, 11:59pm on Gradescope
- Homework 1 is posted on [DSC40b.com](https://dsc40b.com)
  - Due Wed, 11:59pm on Gradescope
  - **Hand-written, submit pdf.**
    - **If not, warning for the first time,**
    - **- 25% of your grade for the second time,**
    - **- 50% of your grade for the third time,**
    - **- 100% for more.**



<https://webclicker.web.app/>

NAMWUZ

**Steps:**

1. Go to a link above
2. Code: NAMWUZ
3. Make sure to use your UCSD email address (i.e., @[ucsd.edu](mailto:ucsd.edu))
4. Use quest/public wifi please.
5. Answer the questions when I active the poll.
6. Do not worry if it does not work today. The first class does not count. We will figure it out eventually.

<https://webclicker.web.app/> **NAMWUZ**

**What is your DSC30 (Similar) status? :)**

A: Already took it

B: Took a similar class

C: Taking it this quarter

D: Still need to take it

E: Something else



***Last Time: Measuring  
Efficiency by Timing***

```
[4]: numbers = range(1000)
```

```
[5]: %%time  
sum (numbers)
```

CPU times: user 30  $\mu$ s, sys: 0 ns, total: 30  $\mu$ s  
Wall time: 34.3  $\mu$ s

```
[5]: 499500
```

```
[6]: %%timeit  
sum(numbers)
```

9.96  $\mu$ s  $\pm$  3.79 ns per loop (mean  $\pm$  std. dev. of 7 runs, 100,000 loops each)

## ***Disadvantages of Timing***

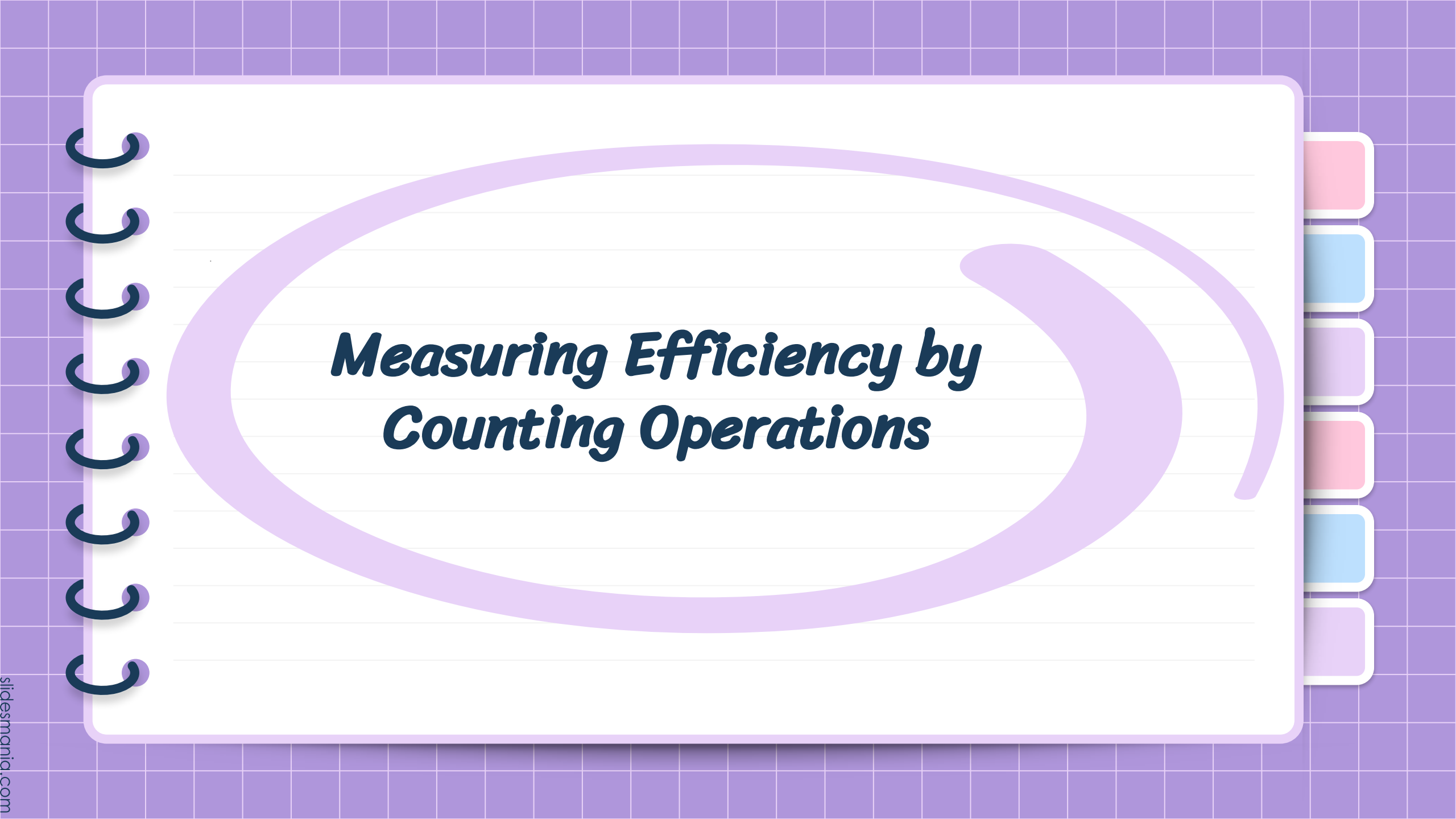
1. Time depends on the computer.

## ***Disadvantages of Timing***

1. Time depends on the computer.
2. Depends on the particular input, too.

## ***Disadvantages of Timing***

1. Time depends on the computer.
2. Depends on the particular input, too.
3. One timing doesn't tell us how algorithm **scales**.



# ***Measuring Efficiency by Counting Operations***



## ***Approach #2: Time Complexity Analysis***

- Determine efficiency of code **without** running it.
- **Idea**: find a formula for time taken as a function of input size.

$T(n)$  -> gives back "time"

## *Advantages of Time Complexity*

1. Doesn't depend on the computer.
2. Reveals which inputs are "hard", which are "easy".
3. Tells us how algorithm **scales**.

## ***Exercise***

Write a function **mean** which takes in a NumPy array of floats and outputs their mean (***without*** a built-in function).

## ***Solution***

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total = total + x  
    return total / n
```

# *Time Complexity Analysis*

- How long does it take mean to run on an array of size  $n$ ?
  - Call this  $T(n)$ .
- We want a **formula** for  $T(n)$ .

# Counting Basic Operations

- Assume certain *basic* operations (like adding two numbers) take a **constant** amount of time.
  - $x + y$  doesn't take more time if *numbers* is bigger.
  - So  $x + y$  takes "constant time"
  - Compare to **sum**(numbers) . **Not** a basic operation.
- **Idea**: Count the number of **basic** operations. This is a *measure* of time.

## Exercise

- What is the complexity for each operation?
  - accessing an element: `arr[i]`
  - asking for the length: `len(arr)`
  - finding the max: `max(arr)`

**A: Constant**

**B:  $\log n$**

**C: Linear (proportional to  $n$ )**

**D: Something else**

## ***Exercise***

- What is the complexity for each operation?
  - accessing an element: `arr[i]` -> Constant
  - asking for the length: `len(arr)` -> Constant
  - finding the max: `max(arr)` -> Linear



## *Basic Operations with Arrays*

- We'll assume that these operations on NumPy arrays take **constant** time.
  - accessing an element: `arr[i]`
  - asking for the length: `len(arr)`

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
|------------|-------------|

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      |             |

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| ?          |             |

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| $c_2$      | ?           |

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $C_1$      | 1           |
| $C_2$      | 1           |
| ?          |             |

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| $c_2$      | 1           |
| $c_5$      | ?           |

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $C_1$      | 1           |
| $C_2$      | 1           |
| ?          |             |
| $C_5$      | 1           |



## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $C_1$      | 1           |
| $C_2$      | 1           |
| $C_4$      | ?           |
| $C_5$      | 1           |

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $C_1$      | 1           |
| $C_2$      | 1           |
| ?          |             |
| $C_4$      | n           |
| $C_5$      | 1           |

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $C_1$      | 1           |
| $C_2$      | 1           |
| $C_3$      | ?           |
| $C_4$      | n           |
| $C_5$      | 1           |

## Example

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $C_1$      | 1           |
| $C_2$      | 1           |
| $C_3$      | $n+1$       |
| $C_4$      | $n$         |
| $C_5$      | 1           |

**Example**  $T(n) = C_1 + C_2 + C_5 + n C_4 + (n+1) C_3$

```
def mean(numbers):  
    total = 0  
    n = len(numbers)  
    for x in numbers:  
        total += x  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $C_1$      | 1           |
| $C_2$      | 1           |
| $C_3$      | $n+1$       |
| $C_4$      | $n$         |
| $C_5$      | 1           |

## ***Example: mean***

- **Total time:**

$$\begin{aligned} T(n) &= c_3(n + 1) + c_4n + (c_1 + c_2 + c_5) \\ &= (c_3 + c_4)n + (c_1 + c_2 + c_3 + c_5) \end{aligned}$$

- “*Forgetting*” constants, lower-order terms with “Big-Theta”:  $T(n) = \Theta(n)$ .
- $\Theta(n)$  is the **time complexity** of the algorithm.

## *Main Idea*

Forgetting constant, lower order terms allows us to focus on how the algorithm **scales**, *independent* of which computer we run it on.

***Careful!***

**Not always** the case that a single line of code takes constant time per execution!



## Example

```
def mean_2(numbers):  
    total = sum(numbers)  
    n = len(numbers)  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| ?          | ?           |

## Example

```
def mean_2(numbers):  
    total = sum(numbers)  
    n = len(numbers)  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| ?          | ?           |
| $c_3$      | 1           |

## Example

```
def mean_2(numbers):  
    total = sum(numbers)  
    n = len(numbers)  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| ?          | ?           |
| $c_2$      | 1           |
| $c_3$      | 1           |

## Example

```
def mean_2(numbers):  
    total = sum(numbers)  
    n = len(numbers)  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| ?          | 1           |
| $c_2$      | 1           |
| $c_3$      | 1           |

## Example

```
def mean_2(numbers):  
    total = sum(numbers)  
    n = len(numbers)  
    return total / n
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1 n$    | 1           |
| $c_2$      | 1           |
| $c_3$      | 1           |

## ***Example: mean\_2***

- **Total time:**

$$T(n) = c_1n + (c_2 + c_3)$$

- “Forgetting” constants, lower-order terms with “Big-Theta”:

$$T(n) = \Theta(n).$$

## ***Exercise***

- Write an algorithm for finding the **maximum** of an array of  $n$  numbers.
- What is its time complexity?

```
def maximum(numbers):
    current_max = -float('inf')
    for x in numbers:
        if x > current_max:
            current_max = x
    return current_max
```

[illegible]



```
def maximum(numbers):  
    current_max = -float('inf')  
    for x in numbers:  
        if x > current_max:  
            current_max = x  
    return current_max
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |

```
def maximum(numbers):
    current_max = -float('inf')
    for x in numbers:
        if x > current_max:
            current_max = x
    return current_max
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| $c_2$      | $n + 1$     |
| ?          | ?           |

```
def maximum(numbers):
    current_max = -float('inf')
    for x in numbers:
        if x > current_max:
            current_max = x
    return current_max
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| $c_2$      | $n + 1$     |
| ?          | ?           |
| $c_3$      | 1           |

```
def maximum(numbers):
    current_max = -float('inf')
    for x in numbers:
        if x > current_max:
            current_max = x
    return current_max
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| $c_2$      | $n + 1$     |
| $c_4$      | ?           |
| $c_3$      | 1           |

```
def maximum(numbers):
    current_max = -float('inf')
    for x in numbers:
        if x > current_max:
            current_max = x
    return current_max
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| $c_2$      | $n + 1$     |
| $c_4$      | $n$         |
| ?          | ?           |
| $c_3$      | 1           |

```
def maximum(numbers):
    current_max = -float('inf')
    for x in numbers:
        if x > current_max:
            current_max = x
    return current_max
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| $c_2$      | $n + 1$     |
| $c_4$      | $n$         |
| $c_5$      | ?           |
| $c_3$      | 1           |

```
def maximum(numbers):
    current_max = -float('inf')
    for x in numbers:
        if x > current_max:
            current_max = x
    return current_max
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| $c_2$      | $n + 1$     |
| $c_4$      | $n$         |
| $c_5$      | $\leq n$    |
| $c_3$      | 1           |

```
def maximum(numbers):
    current_max = -float('inf')
    for x in numbers:
        if x > current_max:
            current_max = x
    return current_max
```

| Time/exec. | # of execs. |
|------------|-------------|
| $c_1$      | 1           |
| $c_2$      | $n + 1$     |
| $c_4$      | $n$         |
| $c_5$      | $\leq n$    |
| $c_3$      | 1           |

$$T(n) = \Theta(n).$$



## *Main Idea*

Using Big-Theta allows us not to worry about **exactly** how many times each line runs.

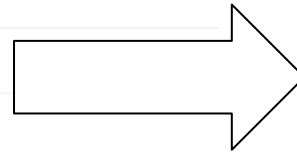
## *Remaining Questions*

- What if the code is more complex?
  - For example, **nested loops**.
- What is this notation anyways?



# ***Analyzing Nested Loops***

## ***Nested Loops. Example 1: Interview Problem***



Given the diameters of  $n$  snowballs, what is the **tallest** snowman you can make using **exactly two** snowballs?

## ***Exercise***

- What is the time complexity of the **brute force** solution?
- **Bonus (think, not vote)**: what is the **best possible** time complexity of any solution?

**A: Constant**

**B: logarithmic**

**C: Linear**

**D: Quadratic**

**E: Something else**

## ***The Brute Force Solution***

- Loop through all possible (ordered) pairs.
  - *How many are there?*
- Check height of each.
- Keep the **best**.

***How many ordered pairs?***

*How many ordered pairs?*

- $N * (N - 1) = N^2 - N$



```
def tallest_snowman(heights):  
    max_height = -float('inf')  
    n = len(heights)  
    for i in range(n):  
        for j in range(n):  
            if i == j:  
                continue  
            height = heights[i] + heights[j]  
            if height > max_height:  
                max_height = height  
    return max_height
```

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

**C**

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

C

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

C

C

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

C

C

C

C

C

C

C

C

C

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

1

C

1

C

?

C

C

C

C

C

C

C

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

| Time/exec. | # of execs. |
|------------|-------------|
| C          | 1           |
| C          | 1           |
| C          | n + 1       |
| C          |             |
| C          | ?           |
| C          |             |
| C          |             |
| C          |             |
| C          |             |

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

1

C

1

C

$n + 1$

C

C

$n * n$

C

?

C

C

C

C



```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

1

C

1

C

$n + 1$

C

$n * n$

C

n

C

?

C

C

C

C

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

1

C

1

C

$n + 1$

C

$n * n$

C

n

C

$n * n - n$

C

?

C

C

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

1

C

1

C

$n + 1$

C

C

$n * n$

C

n

C

$n * n - n$

C

$n * n - n$

C

?

C

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

1

C

1

C

$n + 1$

C

$n * n$

C

$n$

C

$n * n - n$

C

$n * n - n$

C

$\leq n * n - n$

C

?

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

1

C

1

C

$n + 1$

C

?

C

$n * n$

C

$n$

C

$n * n - n$

C

$n * n - n$

C

$\leq n * n - n$

C

1

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**A: n**

**B: n + 1**

**C: n \* n**

**D: Som.else**

| Time/exec. | # of execs.  |
|------------|--------------|
| C          | 1            |
| C          | 1            |
| C          | n + 1        |
| C          | ?            |
| C          | n * n        |
| C          | n            |
| C          | n * n - n    |
| C          | n * n - n    |
| C          | <= n * n - n |
| C          | 1            |

```
def tallest_snowman(heights):
```

```
    max_height = -float('inf')
```

```
    n = len(heights)
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if i == j:
```

```
                continue
```

```
            height = heights[i] + heights[j]
```

```
            if height > max_height:
```

```
                max_height = height
```

```
    return max_height
```

**Time/exec.**

**# of execs.**

C

1

C

1

C

$n + 1$

C

$n(n+1)$

C

$n * n$

C

$n$

C

$n * n - n$

C

$n * n - n$

C

$\leq n * n - n$

C

1

## *Time Complexity*

- Time complexity of this is  $\Theta(n^2)$ .
- TODO: Can we do better?
- **Note**: this algorithm considers each pair of snowballs **twice**.
- We'll fix that in a moment.



## *First: A shortcut*

- Making a table is getting tedious.
- Usually, find a chunk that **dominates** time complexity; i.e., yields the leading term of  $T(n)$ .

## *A Shortcut*

- Assume each line takes constant time to execute *once*.
- To determine the overall time complexity:
  1. Find the line that is execute most.
  2. Count how many times it is executed.

## Shortcut for the Brute Force Solution

```
for i in range(n):  
    for j in range(n):  
        height = heights[i] + heights[j] # <- count execs.
```

- On outer iter. # 1, inner body runs \_\_\_\_\_ times.

## Shortcut for the Brute Force Solution

```
for i in range(n):  
    for j in range(n):  
        height = heights[i] + heights[j] # <- count execs.
```

- On outer iter. # 1, inner body runs \_\_\_\_\_<sup>n</sup>\_\_\_\_\_ times.

## Shortcut for the Brute Force Solution

```
for i in range(n):  
    for j in range(n):  
        height = heights[i] + heights[j] # <- count execs.
```

- On outer iter. # 1, inner body runs \_\_\_\_\_n\_\_\_\_\_ times.
- On outer iter. # 2, inner body runs \_\_\_\_\_times.
- On outer iter. #  $\alpha$ , inner body runs \_\_\_\_\_times.
- The outer loop runs \_\_\_\_\_ times.
- Total number of executions: \_\_\_\_\_

## Shortcut for the Brute Force Solution

```
for i in range(n):  
    for j in range(n):  
        height = heights[i] + heights[j] # <- count execs.
```

- On outer iter. # 1, inner body runs \_\_\_\_\_n\_\_\_\_\_ times.
- On outer iter. # 2, inner body runs \_\_\_\_\_n\_\_\_\_\_times.
- On outer iter. #  $\alpha$ , inner body runs \_\_\_\_\_times.
- The outer loop runs \_\_\_\_\_ times.
- Total number of executions: \_\_\_\_\_

## Shortcut for the Brute Force Solution

```
for i in range(n):  
    for j in range(n):  
        height = heights[i] + heights[j] # <- count execs.
```

- On outer iter. # 1, inner body runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- On outer iter. # 2, inner body runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- On outer iter. #  $\alpha$ , inner body runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- The outer loop runs \_\_\_\_\_ times.
- Total number of executions: \_\_\_\_\_

## Shortcut for the Brute Force Solution

```
for i in range(n):  
    for j in range(n):  
        height = heights[i] + heights[j] # <- count execs.
```

- On outer iter. # 1, inner body runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- On outer iter. # 2, inner body runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- On outer iter. #  $\alpha$ , inner body runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- The outer loop runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- Total number of executions: \_\_\_\_\_



## Shortcut for the Brute Force Solution

```
for i in range(n):  
    for j in range(n):  
        height = heights[i] + heights[j] # <- count execs.
```

- On outer iter. # 1, inner body runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- On outer iter. # 2, inner body runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- On outer iter. #  $\alpha$ , inner body runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- The outer loop runs \_\_\_\_\_  $n$  \_\_\_\_\_ times.
- Total number of executions:  $\underbrace{n + n + \dots + n}_{n} = n^2$

## ***Example 2: The Median***

- **Given:** real numbers  $x_1, \dots, x_n$ .
- **Compute:**  $h$  minimizing the **total absolute loss**

$$R(h) = \sum_{i=1} |x_i - h|$$

## ***Example 2: The Median***

- **Solution:** the **median**.
- That is, a **middle** number.
- But how do we actually **compute** a median?

## A Strategy

- **Recall**: one of  $x_1, \dots, x_n$  must be a median.
- **Idea**: compute  $R(x_1), R(x_2), \dots, R(x_n)$ , return  $x_i$  that gives the smallest result.

$$R(h) = \sum_{i=1}^n |x_i - h|$$

- Basically a **brute force** approach.

## ***Exercise***

- What is the time complexity of this brute force approach?
- How long will it take to run on an input of size 10,000?

```
def median(numbers):  
    min_h = None  
    min_value = float('inf')  
    for h in numbers:  
        total_abs_loss = 0  
        for x in numbers:  
            total_abs_loss += abs(x - h)  
        if total_abs_loss < min_value:  
            min_value = total_abs_loss  
            min_h = h  
    return min_h
```

```
def median(numbers):
```

```
    min_h = None
```

```
    min_value = float('inf')
```

```
    for h in numbers:
```

```
        total_abs_loss = 0
```

```
        for x in numbers:
```

```
            total_abs_loss += abs(x - h)
```

```
        if total_abs_loss < min_value:
```

```
            min_value = total_abs_loss
```

```
            min_h = h
```

```
    return min_h
```

**What is the complexity for each line of code?**

```
def median(numbers):
```

```
    min_h = None
```

```
    min_value = float('inf')
```

```
    for h in numbers:
```

```
        total_abs_loss = 0
```

```
        for x in numbers:
```

```
            total_abs_loss += abs(x - h)
```

```
        if total_abs_loss < min_value:
```

```
            min_value = total_abs_loss
```

```
            min_h = h
```

```
    return min_h
```

**What is the complexity for each line of code?**

**What line executes the most?**



```
def median(numbers):
```

```
    min_h = None
```

```
    min_value = float('inf')
```

```
    for h in numbers:
```

```
        total_abs_loss = 0
```

```
        for x in numbers:
```

```
            total_abs_loss += abs(x - h)
```

$n^2$

```
        if total_abs_loss < min_value:
```

```
            min_value = total_abs_loss
```

```
            min_h = h
```

```
    return min_h
```

**What is the complexity for each line of code?**

**What line executes the most?**

**$T(n) = \Theta(n^2)$**

## *The Median*

- The brute force approach has  $\Theta(n^2)$  time complexity.
- **TODO:** Is there a better algorithm?

## *The Median*

- The brute force approach has  $\Theta(n^2)$  time complexity.
- **TODO:** Is there a better algorithm?
  - It turns out, you can find the median in **linear** time.  
(*expected*)

# *The Median*

```
numbers = list(range(10000))
```

```
%time median(numbers)
```



```
CPU times: user 4.55 s, sys: 0 ns, total: 4.55 s
```

```
Wall time: 4.55 s
```

```
4999
```

# *The Median*

```
numbers = list(range(10000))
```

```
%time median(numbers)
```



```
CPU times: user 4.55 s, sys: 0 ns, total: 4.55 s
```

```
Wall time: 4.55 s
```

```
4999
```

```
%time magic_median(numbers)
```

```
CPU times: user 5.42 ms, sys: 22 µs, total: 5.44 ms
```

```
Wall time: 5.04 ms
```

## Careful!

- **Not every nested loop has  $\Theta(n^2)$  time complexity!**
- In general, if:
  - outer loop iterates  $a$  times;
  - inner loop iterates  $b$  times for each outer loop iteration
    - We are assuming here that the number of inner loop iterations doesn't depend on which outer loop iteration we're in!
  - then the innermost loop body is executed  $a \times b$  times.

```
for x in range(n):  
    for y in range(n**2):  
        print(x + y)
```

## Example 3

```
def foo(n):  
    for x in range(n):  
        for y in range(60000):  
            print(x + y)
```

**Time complexity?**

**A: Constant**

**B:  $n$**

**C:  $n \log n$**

**D:  $n^2$**

## ***Example 4***

```
def f(n):  
    for i in range(3*n**3 + 5*n**2 - 100):  
        for j in range(n**5, n**6):  
            print(i, j)
```



## Example 4

```
def f(n):  
    for i in range(3*n**3 + 5*n**2 - 100):  
        for j in range(n**5, n**6):  
            print(i, j)
```

**Ans:**

$\Theta(n^9)$



# *Thank you!*

**Do you have any questions?**

CampusWire!